



staunton

Chess diagrams, games and tournament tables for Typst

<https://github.com/ndg6/staunton>

User manual · package version 0.1.0

Guide

1	Introduction	3
1.1	Installing and Importing	3
1.2	How to read this manual	3
1.3	The Name	3
1.4	Acknowledgements	3
2	The Board	4
2.1	Labels	4
2.2	Highlights	4
2.3	Arrows and the Grid	5
2.4	Coordinates and Non-Square Boards	5
2.5	Sizing	6
2.6	Colours	6
2.7	Flip	6
2.8	Piece Sets and Fonts	7
3	Diagrams	7
3.1	Boards are not Figures	8
4	Positions	8
5	Games (PGN)	9
5.1	Locators	10
5.2	Playing Moves onto a Position	10
5.3	Notation Output	10
5.4	Exporting FEN	13
5.5	Drawing Annotations in PGNs	14
5.6	PGN Handling	14
5.7	Errors	15
6	Tournament Tables	15
7	Outlines and References	16
8	Document-Wide Defaults	16
8.1	Language	17
8.2	PGN Handling	17
9	HTML Export	18

API Reference

10	Common Parameters	19
10.1	Argument Value Shapes	19
10.2	Board Style Options	19
10.3	Diagram, Table, Language, and PGN- Handling Options	20
11	Main Functions	21
11.1	Boards and Diagrams	21
11.2	Positions	23
11.3	Games (PGN)	25
11.4	Annotate and Build	28
11.5	Notation	29
11.6	Tournament Tables	30
11.7	Outlines and References	34
11.8	Document Defaults	35
12	Behind the Scenes	38
12.1	Engine and Position Internals	38
12.2	SAN	39
12.3	Tournament Data	40
12.4	Rendering and Annotation	41
12.5	Coordinates	42
12.6	Pieces and variants	44
12.7	Localization	46

1 Introduction

Package **staunton** aims to provide a complete, convenient but also flexible solution for chess publications. It provides a full set of features, including:

- **boards and diagrams** — pure boards with labels, highlights, arrows, an optional grid, flexible sizing, custom colours, and bundled SVG piece sets (or a Unicode fallback); and building on that diagrams with captions, figure counters, and referenceable labels;
- **games from PGN** — a sophisticated parser creates single games or an array of games, from which you create positions by “locators” (mainline and variations), move play-out, and FEN export;
- **move notation** — from parsed games you get move text output with localized piece letters, figurine glyphs, NAGs, comments and diagrams embedded inline;
- **tournament tables** — we can create standings, cross-tables and progress charts from a PGN’s results, by player or by team;
- **outlines and references** — diagrams and tables get their own counters and lists;
- **document-wide styling and localization** (six languages, easily extended);
- **limited HTML export** — notation, tables, outlines, references and captioned figures become native HTML, with boards and diagrams embedded as inline SVG (see Section 9).

```
#chess-diagram(  
  "r1bqkbnr/pppp1ppp/2n5/4p3/4P3/5N2/  
PPPP1PPP/RNBQKB1R",  
  caption: [The Ruy Lopez, three moves  
in.],  
  size: 4cm,  
)
```

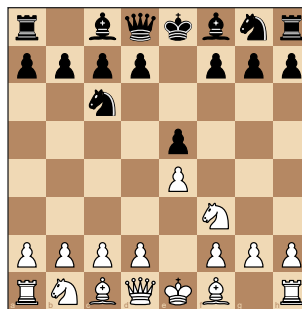


Diagram 1: The Ruy Lopez, three moves in.

This manual is bottom-up. The **board** is the drawing primitive; a **diagram** wraps a board in a referenceable figure; a **position** is the data a board draws; **games** add PGN, notation, and play; then come **tournament tables**, **outlines and references**, and the **document-wide defaults**.

1.1 Installing and Importing

staunton is a Typst package. Import its public API once and every function in this manual is in scope:

```
#import "@preview/staunton:0.1.0": *
```

1.2 How to read this manual

In every framed example, the left side is **the code you type** and the right side is **exactly what it renders** — the manual compiles its own examples, so the two can never disagree.

1.3 The Name

Typst package **staunton** is named in honour of **Howard Staunton** (c. 1810–1874): a leading chess master of his day, organiser of the first international tournament (London, 1851), a chess author and publisher, and the namesake of the standardised **Staunton pattern** chessmen — still the tournament standard.

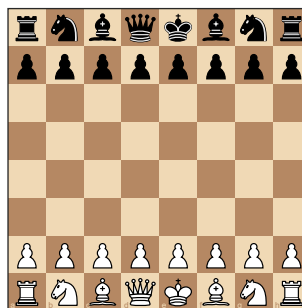
1.4 Acknowledgements

The Typst package **boards-n-pieces** was an inspiration for some of **staunton**’s features. This package and its manual were developed with assistance from Claude (Opus 4.8) by Anthropic.

2 The Board

`board(source, ...)` draws **just the board** — no caption, no figure — and is the primitive every diagram builds on. `source` is one of: a **FEN string**; a **position** (from `position(...)` or `parse-fen(...)`); or a bare **squares** dict (square name → (kind, color)).

```
#board(
  "rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/
  RNBQKBNR",
  size: 4cm,
)
```



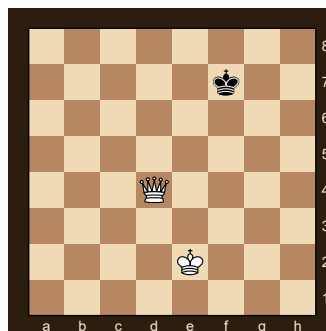
`board` is the variant-agnostic primitive; `chess-board` is the standard-chess sugar over it (same rendering, but it documents the variant and rejects a non-standard source). Use `board` inline in text or inside your own layout; reach for a **diagram** (next chapter) when you want a captioned, referenceable figure.

The rest of this chapter covers the board's drawing options: labels, highlights, arrows, the grid, coordinates, size, colours, orientation, and piece sets — all of which a `chess-diagram` accepts too.

2.1 Labels

`label-mode` chooses how labels for files and ranks are drawn — "on-square" (default, tucked into the corner of the squares), "outside" (a gutter strip), or "border" (a themed band, styled by `border-theme`). `labels: false` suppresses labels completely.

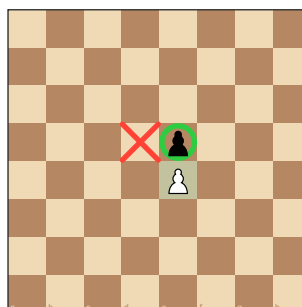
```
#board(
  "8/5k2/8/8/3Q4/8/4K3/8",
  label-mode: "border",
  border-theme: "brown",
  size: 3.8cm,
)
```



2.2 Highlights

`highlight` marks squares; each entry is a square name (drawn with `highlight-shape`, default "filled"), a (square, color) pair, or a dict (square: .., shape: .., color: ..) where shape is "filled", "cross", or "circle". By convention a **cross** marks an empty square.

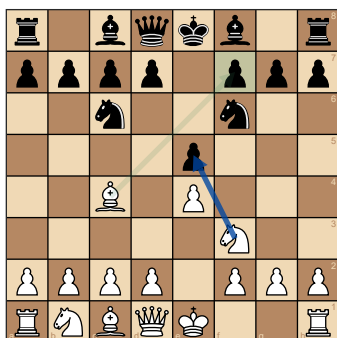
```
#board(
  "8/8/8/4p3/4P3/8/8/8",
  highlight: (
    "e4",
    (square: "e5", shape: "circle"),
    (square: "d5", shape: "cross"),
  ),
  size: 4cm,
)
```



2.3 Arrows and the Grid

As the name suggests `arrows` draws arrows on the board; each entry is a `(from, to)` or `(from, to, color)` tuple, or a dict `(from: ..., to: ..., color: ...)`. A missing color uses `arrow-color`. Arrows scale with the board and flip with it. A `grid: true` overlay draws thin lines between the squares.

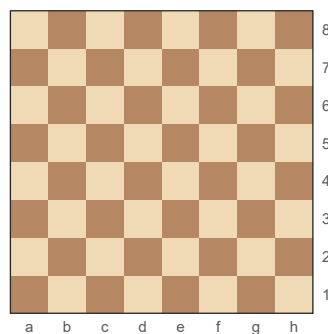
```
#board(
  "r1bqkb1r/pppp1ppp/2n2n2/4p3/2B1P3/5N2/PPPP1PPP/RNBQK2R",
  grid: true,
  highlight: ("f7",),
  arrows: (("c4", "f7"), ("f3", "e5", rgb(0, 70, 160, 200))),
  size: 4.4cm,
)
```



2.4 Coordinates and Non-Square Boards

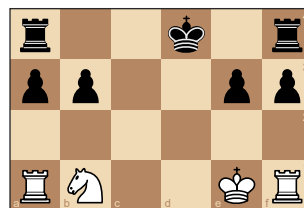
At least in standard western chess, Files run `a`, `b`, ... and ranks `1`, `2`, ...; `a1` is the dark square in the lower-left corner, `h8` the upper-right. Square names are case-insensitive (`"E4"` = `"e4"`).

```
#board(
  "8/8/8/8/8/8/8/8",
  label-mode: "outside",
  size: 4cm,
)
```



But boards are **not** tied to 8×8. A `position` built from the string form (next chapter) counts its own columns and rows, and the renderer draws whatever geometry it is given — files and ranks extend as far as the board needs, and the cells stay square while the board itself becomes rectangular:

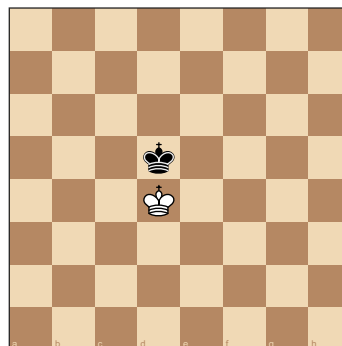
```
#board(
  position(
    "r..k.r",
    "pp..pp",
    ".....",
    "RN..KR",
  ),
  size: 4cm,
)
```



2.5 Sizing

The default board size suits an A4 two-column layout. A board reads the available space at its insertion point via `layout`, so it adapts to any column or page size without being told the geometry, and shrinks to fit if asked for more than fits. `size` may be a `length`, a `ratio` of the available width, or `auto`:

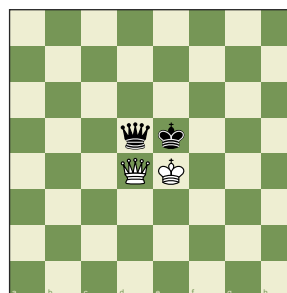
```
#board(
  "8/8/8/3k4/3K4/8/8/8",
  size: 60%, // of the available
width
)
```



2.6 Colours

`light` and `dark` set the two square colours:

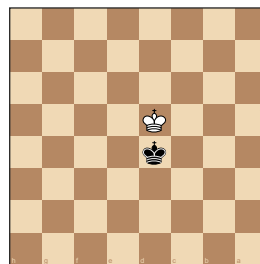
```
#board(
  "8/8/8/3qk3/3QK3/8/8/8",
  light: rgb("#eeeeed2"),
  dark: rgb("#769656"),
  size: 3.8cm,
)
```



2.7 Flip

`flip: true` shows the board from Black's side; labels, highlights and arrows all flip with it. Orientation is a **per-board** choice — `flip` is the one setting that cannot be made a document default (see **Document-wide style**).

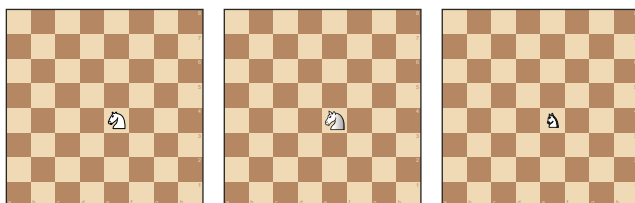
```
#board("8/8/8/3k4/3K4/8/8/8", flip:
true, size: 3.4cm)
```



2.8 Piece Sets and Fonts

Pieces are drawn from bundled **SVG piece sets**. Two ship with the package, both GPLv2+: `cburnett` (default) and `merida`. Choose one with `piece-set:` per board, or document-wide with `set-piece-set` (see Section 8):

```
#grid(columns: 3, gutter: 8pt, align: bottom,
  board("8/8/8/8/4N3/8/8/8", size: 2.6cm, piece-set: "cburnett"),
  board("8/8/8/8/4N3/8/8/8", size: 2.6cm, piece-set: "merida"),
  board("8/8/8/8/4N3/8/8/8", size: 2.6cm, piece-set: "unicode"),
)
```



The `renderer` accepts **any** set name and loads `src/assets/piece_sets/<name>/{w,b}{K,Q,R,B,N,P}.svg` on demand, so you can add a set by dropping such a folder into your copy and passing its name. `piece-set:` (or `none`) selects the glyph fallback `"unicode"`

— solid Unicode chess glyphs distinguished by fill and a contrasting stroke; it needs a font carrying them.

The rank/file **labels** are drawn in their own sans-serif, set by the `label-font` board option (a family or a fallback list), independent of the document font. The default is `("Arial", "DejaVu Sans Mono")` — Arial on Windows/macOS, falling back to Typst’s always-embedded mono — so a stock install draws labels without “unknown font family” warnings. Override it like any board default:

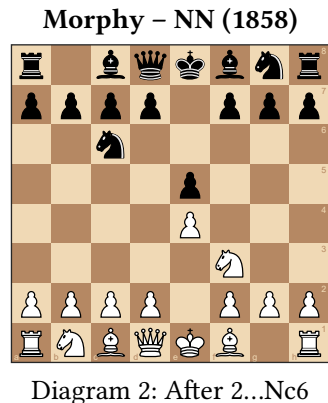
```
#set-board-defaults(label-font: "Segoe UI") // or a list, e.g. ("Helvetica",
"DejaVu Sans Mono")
```

3 Diagrams

`chess-diagram(source, ..)` wraps a board in a `#figure` (kind `"chess"`), so — unlike a bare `board` — it is captioned, counted, referenceable, and listed by an outline. `source` is the same FEN / position / squares the board takes, and it accepts every board option from the previous chapter.

A diagram carries two optional labels: a **game-info** line above the board (drawn automatically as `"<White> — <Black> (<Year>)"` when both players are known) and the figure **caption** below it.

```
#chess-diagram(
  "r1bqkbnr/pppp1ppp/2n5/4p3/4P3/5N2/
  PPPP1PPP/RNBQKB1R",
  white: "Morphy", black: "NN", year:
  1858,
  caption: [After 2...Nc6],
  size: 4.2cm,
)
```



`chess-diagram` is the standard-chess sugar over the generic `diagram`; both take the same source and overrides as `board`. Extra named arguments are forwarded to `figure` (e.g. `placement: top`).

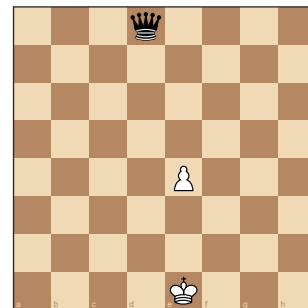
3.1 Boards are not Figures

A bare `board` is plain content: it has **no** caption, **no** figure counter, does **not** resolve `@`-references, and is **not** listed by `chess-diagram-outline`. Only a **diagram** is a figure. So draw a `board` for an inline or decorative position, and a `chess-diagram` whenever you want to caption it, cross-reference it (`@label`), or list it — see **Outlines and references**.

4 Positions

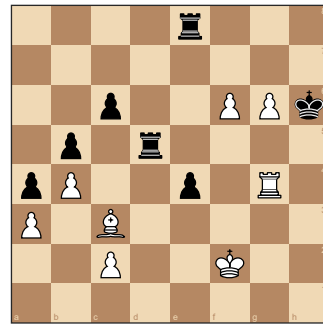
`position(...)` builds the data model — “which piece stands on which square” — that a board or diagram draws. It accepts a FEN string (auto-detected), a **squares dict**, or a **string form**. In a squares dict the piece can be a long name, a kind abbreviation, or a bare letter (UPPER = white, lower = black):

```
#chess-diagram(
  position((
    e1: (kind: "king", color: "white"), // long
    d8: (kind: "q", color: "black"),    // kind
    e4: "P",                           // letter
  )),
  size: 4cm,
)
```



The **string form** reads like the board itself — first line is the TOP rank, `.` is empty. Pass it as a raw block, as below — the most legible, least error-prone way, with no per-line quotes or commas (several row strings work too). It is rectangular-only (this is what lets a board be non-8×8) and rejects characters that aren’t a valid piece abbreviation or `.`:


```
#chess-diagram(
  position(`
    ....r...
    .....
    ..p..PPk
    .p.r....
    pP..p.R.
    P.B.....
    ..P..K..
    .....
    `),
  size: 4.2cm,
)
```



`position` returns a dict (variant, cols, rows, squares, turn, castling, ; `parse-fen` returns en-passant, halfmove, fullmove) the same shape. The `cols` / `rows` are counted from the string form (otherwise the 8×8 default).

5 Games (PGN)

The predominant form of the distribution and publications of chess games (atleast for western chess) are *PGN files*. PGN stands for **Portable Game Notation** and is a text format for chess games. It is human-readable, and can be parsed by chess software. PGN files contain the moves of a game, along with metadata such as player names, event, date, and result. PGN files can contain just one or many games.

The `parse-pgn` function returns an **array of games**; `.first()` takes the first one. Read an external file with `read` in your own file, or pass an inline raw block. The examples below assume an already parsed game is in scope:

```
#let game = parse-pgn(read("game.pgn")).first()
```

Parsing is **lazy**: the roster (`tags`), the `result` , and the verbatim `movetext-raw` are extracted cheaply; the move tree is built on demand by `movetext(game)` ; the move parser / generator engine runs only when you ask for a position. So a tournament file read only for results and never tokenises movetext.

`chess-notation(game)` renders the moves (as text) the game already holds, and `diagram-after(game, loc)` renders a **diagram** (a referenceable `#figure`, like `chess-diagram`) of the position at a locator:

```
#chess-notation(game)
```

```
1.e4 e5 2.Nf3 Nc6 3.Bb5 a6 4.Ba4 Nf6 5.O-O Be7 6.Re1 b5 7.Bb3 d6 8.c3 O-O
```

```
#diagram-after(game, "3w", size: 4cm)
```

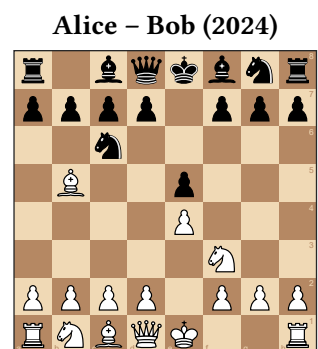


Diagram 5: Position after move 3. Bb5

5.1 Locators

A **locator** addresses one position in a game. The simple form is a string — `"30w"` / `"30b"`, the position after White's / Black's 30th **mainline** move. `position-after(game, loc)`, `diagram-after(game, loc, ..)`, and `to-fen(game, locator: ..)` all take this simple string form.

To address a move **inside a variation** (a PGN 'Recursive Annotation Variation' or RAV), pass a **path** dict instead: `(line: (..hops..), at: "<final move>")`. Each hop is `(at: "<move>", into: <n>)` — branch off at mainline move `at`, descending `into` that move's variation number `n` (**0-based**: `0` is the first variation recorded at that move, `1` the second, ...). The top-level `at` is where you stop within the line you reached:

```
#let g = parse-pgn(
  "[White \"V\"][Black \"T\"] 1. e4 (1.
d4 d5 2. c4) e5 *",
).first()
// into variation 0 at White's move 1
// (the
// 1.d4 line), position after 2.c4:
#diagram-after(
  g,
  (line: ((at: "1w", into: 0),), at:
"2w"),
  size: 3.4cm,
)
```



Diagram 6: Position after move 2. c4

Two easy traps:

- **The trailing comma.** A single-hop path is written `((at: "1w", into: 0),)` — the comma makes it a one-element **array** of hops. Without it, Typst reads a lone parenthesised dict and the locator is malformed.
- **Mainline is the fast path.** The string form (equivalently an empty `line: ()`) indexes a memoised list of mainline positions, so many diagrams off one game stay cheap; the path form is walked move by move.

Addressing a variation that isn't there (`into:` past the recorded count) or a move past the end of its line is a hard error.

5.2 Playing Moves onto a Position

To explore a **new** line, or build a position from a FEN plus some moves, use `chess-moves(source, moves)`. `source` is `none` (the standard start), a FEN string, or a position; `moves` is move text or a SAN array. It resolves each move against the legal moves (illegal/ambiguous is a hard error) and returns the **final** position, never mutating the source:

```
#chess-diagram(
  chess-moves(none, "1. e4 e5 2. Nf3
Nc6 3. Bb5 a6"),
  size: 4cm,
)
```



5.3 Notation Output

For chess publications, notational output of the move text is as important as showing board positions. This output has to be flexible and localisable. While you sometimes want to show move text exactly as it was

recorded in the PGN, you often want to amend and reformat the move text for your own purposes. The `notation(...)` function is the workhorse for this. It takes a game, a move-text string, or a SAN array and produces a formatted move text output. It can localise the piece letters and render figurine glyphs, and it can include or exclude move numbers, results, NAGs, comments, and embedded diagrams.

To support different chess variants in the future `notation(...)` is the **variant-agnostic** primitive and `chess-notation(...)` is the **standard-western-chess** wrapper over it — identical output today, but `chess-notation` fixes the variant and rejects a source of a non-standard `variant`. The split is deliberate and forward-looking: staunton is **variant-forward**, so a future variant gets its own name — a planned `xiangqi-notation`, `shogi-notation`, ... — each a thin wrapper over the same generic `notation` core, while `chess-notation` stays western-chess-specific. The same pairing runs through the package's drawing and notation entry points — `board / chess-board`, `diagram / chess-diagram`, `notation / chess-notation`: reach for the `chess-` name for ordinary chess, and the generic one only when you are deliberately variant-agnostic. Two functions stand slightly apart: `position` is variant-parameterised (the variant rides on its source or `variant:` argument, so there is no separate `chess-position`), and `chess-moves` is chess-only today (the engine does standard chess, so there is no generic `moves`). Both this manual and everyday use favour `chess-notation`.

Either formats move text the game already holds — no engine. Its `source` is a game, a move-text string, or a SAN array; it localises the piece letters and renders figurine glyphs:

```
#chess-notation(game, lang: "de")
```

```
1.e4 e5 2.Sf3 Sc6 3.Lb5 a6 4.La4 Sf6 5.O-O Le7 6.Te1 b5 7.Lb3 d6 8.c3 O-O
```

```
#chess-notation(game, figurine: true)
```

```
1.e4 e5 2.♘f3 ♗c6 3.♙b5 a6 4.♙a4 ♘f6 5.O-O ♗e7 6.♖e1 b5 7.♙b3 d6 8.c3 O-O
```

`from` / `to` restrict output to an **inclusive** slice of moves. Both are the simple `"8b"` / `"12w"` locators; `from` defaults to the first move, `to` to the last:

```
#chess-notation(game, from: "2w", to: "3b")
```

```
2.Nf3 Nc6 3.Bb5 a6
```

`from` / `to` bound a slice of the **mainline**: they are the simple `"8b"` / `"12w"` locators, not the variation **path** form that `diagram-after` takes (rendering variations is a separate control — see the next section). A `from` past the end or a `to` before `from` is a hard error.

Other options: `move-numbers`, `result`, `bold-mainline` (render the mainline moves bold to set them off from variations — see **Variations**), and — for a **game** source — `nags` / `comments`. The last three consult the PGN-handling defaults. Localization substitutes only the piece letters; files, ranks, captures, check marks and `0-0` are untouched.

5.3.1 Variations

By default `notation` renders only the **mainline**. Set `variations: true` (or the `set-pgn-defaults(variations: ...)` document default) to splice the game's variations (RAVs) into the output — in parentheses, correctly numbered: a white-first line reads `3.Bc4 ...`, a black-first line `3...Bc5 ...`, and the resumed mainline move re-shows its number:

```
#chess-notation(
  parse-pgn(
    "1. e4 e5 2. Nf3 Nc6 3. Bb5 (3. Bc4 Bc5) a6 *",
  ).first(),
  variations: true,
)
```

1.e4 e5 2.Nf3 Nc6 3.Bb5 (3.Bc4 Bc5) 3...a6

Variations nest to any depth and honour `nags` / `comments` / `figurine` / `lang` inside the parentheses. `variation-style: "block"` keeps the parentheses but breaks each variation onto its own line, indented one level per nesting depth (a variation that ends on a nested line closes with a `)` on its own line) — the analysis-view layout:

```
#chess-notation(
  parse-pgn(
    "1. e4 (1. d4 d5 (1... Nf6 2. c4)) e5 *",
  ).first(),
  variations: true,
  variation-style: "block",
)
```

```
1.e4
  (1.d4 d5
    (1...Nf6 2.c4)
  )
1...e5
```

To render **one specific** variation on its own, pass `line:` — a path locator (the same `diagram-after` shape, or just its hops array) that descends into the variation. It is numbered from its real branch ply, so it reads exactly as you'd write it in analysis:

```
#let g = parse-pgn(
  "1. e4 e5 2. Nf3 Nc6 (2... d6 3. d4) 3. Bb5 *",
).first()
// the 2...d6 side line, on its own:
#chess-notation(g, line: ((at: "2b", into: 0),))
```

2...d6 3.d4

Each hop is `(at: "<move>", into: <n>)` — nest hops to reach a deeper line. The addressed line's **own** variations follow the `variations` flag. (`from` / `to` stay mainline-only and don't combine with `line`)

5.3.2 Building on a Game: NAGs and Comments

Beyond `nags: true` (which renders the `$n` NAGs a game **already** carries), you can attach NAGs and comments **programmatically** — without editing the PGN — and then render them like any parsed game. `with-nags(game, ..)` and `with-comments(game, ..)` each return a **new** game (the source is never mutated), so they compose:

```
#let g = parse-pgn(
  "1. e4 e5 2. Nf3 Nc6 3. Bb5 (3. Bc4 Bc5) a6 *",
).first()
#chess-notation(
  with-comments(
    with-nags(g, ("3w": "!")),
    (((line: ((at: "3w", into: 0)), at: "3w"), "a sharp try")),
  ),
  variations: true, nags: true, comments: true,
)
```

1.e4 e5 2.Nf3 Nc6 3.Bb5! (3.Bc4 a sharp try Bc5) 3...a6

Moves are addressed the same way everywhere — a **mainline** locator ("3w") or a variation **path** dict — so you can annotate a move **inside** a (nested) variation, not only the mainline. Two input forms:

- a **dict** ("3w": "!", "5b": "\$14") — mainline moves only (dict keys are strings);
- an **array of (locator, value) pairs** — any move, including a path-dict locator (as in the example above).

`with-nags` values are a "\$n" code or a suffix glyph (! ? !! ?? !? ?! , sugar for \$1 – \$6), or an array of those; `with-comments` values are plain-text strings. Both **replace** what was on the move. Comments show only with `comments:` , NAGs only with `nags: true` .

5.3.3 Adding Variations

`with-variation(game, at:, moves:)` grows the tree: it adds a variation as an **alternative** to the move at `at` (a mainline locator or a path dict). `moves` is a PGN movetext fragment — the same syntax `parse-pgn` reads — so one call can carry move numbers (recomputed), nested `()` variations, `$n` NAGs, and `{comments}` ; a plain SAN run like "Bc4 Bc5" is the simplest case:

```
#let g = parse-pgn("1. e4 e5 2. Nf3 Nc6 3. Bb5 a6 *").first()
#chess-notation(
  with-variation(g, at: "3w",
    moves: "3. Bc4 Bc5! (3... Nf6 4. d4) {a sharp alternative}"),
  variations: true, nags: true, comments: true,
)
```

1.e4 e5 2.Nf3 Nc6 3.Bb5 (3.Bc4 Bc5! a sharp alternative (3...Nf6 4.d4)) 3...a6

The variation is **appended** to the move's variations (its index `into` is the previous count — 0 for a move with none yet), so you can address into it afterwards and it composes with `with-nags` / `with-comments` . Together these let you build a whole annotated tree from a bare game, then render or navigate it exactly like a parsed PGN. Moves are **not** checked for legality when added — an illegal move surfaces only if you navigate into the line (`diagram-after`), matching the rest of the lazy model.

5.4 Exporting FEN

`to-fen` is the inverse of `parse-fen` . It serialises a position, or a game at a locator. Standard 8×8 positions round-trip exactly:

```
#raw(to-fen(chess-moves(none, "1. e4 e5 2. Nf3")))
```

```
rnbqkbnr/pppp1ppp/8/4p3/4P3/5N2/PPPP1PPP/RNBQKB1R b KQkq - 1 2
```

5.5 Drawing Annotations in PGNs

PGN comments can carry drawing annotations — `[%cal ...]` for arrows, `[%csl ...]` for highlights. Processing is **off by default**; opt in per call with `annotations: true` (or document-wide with `set-pgn-defaults` — see Section 5.6). The demo game annotates its 2nd move:

```
// move 2: {[%cal Gf3e5] [%csl Re5]}
#diagram-after(game, "2w", annotations:
true, size: 4cm)
```

Alice – Bob (2024)

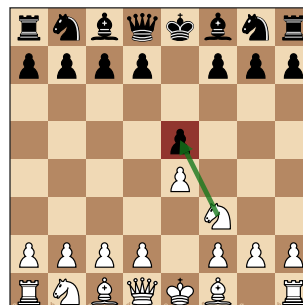


Diagram 8: Position after move 2. Nf3

The color letters (G R Y B O) resolve through the `annotation-colors` board style; annotations merge with any `arrows` / `highlight` you pass explicitly.

With embedded diagrams. The `diagrams` switch (PGN handling) makes `notation(diagrams: true)` splice a board after each move whose comment carries a diagram marker. If that **same** comment also holds `%cal` / `%csl` and `annotations` is on, the spliced board shows those arrows/highlights too — the two switches compose:

```
// 2. Nf3 {[%cal Gf1c4] [%csl Re5] #[After 2.Nf3]} Nc6 ...
#set-pgn-defaults(diagrams: true, annotations: true)
#chess-notation(game) // ...text, then a board after 2.Nf3 with the arrow +
highlight
```

`diagrams` decides **whether** a board appears; `annotations` decides whether it carries the marks. The marker and the `%cal` / `%csl` must be in the **same** move's comment. (Only mainline moves are addressed — `notation` renders the mainline.)

5.6 PGN Handling

The switches above — `annotations`, `nags`, `comments`, `diagrams`, `variations`, `bold-mainline` — form the **PGN-handling** group: they decide how much of a parsed game's embedded extras get interpreted at render time. Parsing itself stays lossless; these only decide what is **processed**, and (except `bold-mainline`) **all default off**. Each is a per-call argument (`auto` → the document default) on `notation` / `diagram-after`, or a document-wide default via `set-pgn-defaults`:

```
#set-pgn-defaults(annotations: true, nags: true, comments: true)
```

key	effect
<code>annotations</code>	<code>%cal</code> / <code>%csl</code> → arrows/highlights on <code>diagram-after</code>
<code>nags</code>	render NAGs (Nf3!, d4±) in <code>notation</code>
<code>comments</code>	include comment prose in <code>notation</code>
<code>diagrams</code>	embed a board in <code>notation</code> after each move marked for one
<code>variations</code>	splice variations (RAVs) into <code>notation</code> , in parentheses
<code>bold-mainline</code>	render <code>notation</code> mainline moves bold (variations stay normal)

`set-chess-defaults` routes these same keys through its umbrella, alongside the board, diagram, table and language buckets — see Section 8.

5.7 Errors

Malformed PGN is a **hard error**: broken tag syntax and stray variation parens fail at parse time; illegal, ambiguous, or unparseable moves fail when the position is navigated. Missing Seven-Tag-Roster tags are tolerated (they default).

6 Tournament Tables

Tournament tables are built from a parsed PGN's roster + results (no engine). The `*-table` renderers produce a captioned, referenceable `#figure` (kind `"chess-table"`). Each works on a list of games, with `by: "player"` or `by: "team"`. The examples use a parsed 4-player round-robin in `games`:

```
#let games = parse-pgn(read("event.pgn"))
```

`standings-table` sorts best-first (score, then tie-breaks, then first appearance):

```
#standings-table(games, by: "player")
```

Pos	Player	Pl	+	=	-	Pts	Bh	SB
1	Alice	3	2	1	0	2½	3½	2½
2	Dave	3	1	2	0	2	4	2½
3	Bob	3	1	0	2	1	5	½
4	Carol	3	0	1	2	½	5½	1

`crosstable-table` renders the round-robin grid (it **requires** a complete round-robin and errors otherwise — use `standings` + `progress` for Swiss/league):

```
#crosstable-table(games, by: "player")
```

#	Player	1	2	3	4	Pts
1	1 Alice		½	1	1	2½
2	2 Dave	½		1	½	2
3	3 Bob	0	0		1	1
4	4 Carol	0	½	0		½

`progress-table` shows the round-by-round running score (needs the `Round` tag):

```
#progress-table(games, by: "player")
```

#	Player	R1	R2	R3	Pts
1	Alice	1 (1)	1 (2)	½ (2½)	2½
2	Dave	½ (½)	1 (1½)	½ (2)	2
3	Bob	0 (0)	0 (0)	1 (1)	1
4	Carol	½ (½)	0 (½)	0 (½)	½

Each renderer takes `caption` (used by refs and the outline), `title` (a heading above the table), `supplement`, and `lang`. The compute functions (`standings`, `crosstable`, `progress`) are also public,

returning plain data for custom layouts. **Team** mode groups games by the `Round = "round.board"` convention into matches.

7 Outlines and References

Diagrams and tables each carry a distinct figure `kind ( "chess" / "chess-table" )`, so they get their own counters, can be **referenced** (`@label` → “Diagram 3” / “Table 2”), and can be **listed separately**:

```
#chess-diagram-outline()    // list of chess diagrams
#chess-table-outline()      // list of tournament tables
#chess-outlines()           // both, diagrams then tables
```

```
#chess-diagram(starting-fen, caption: [Start]) <start>
```

As shown in `@start`, ...

Only figures that carry a **caption** appear in an outline (a caption-less figure is still referenceable but unlisted, matching Typst). Titles are language-aware by default and settable per call (`title: , lang:`) or document-wide (`set-diagram-defaults(outline-title: ..)` — see Section 8). Remember that a bare `board` is not a figure, so it can be neither referenced nor listed — use a `chess-diagram`.

8 Document-Wide Defaults

Package **staunton** features sensible default settings out of the box, you rarely need to adjust them at all. But if you need to, you can achieve that in two ways: per-call arguments overriding the default settings, and **document-wide** defaults that can be set with `set-chess-defaults` or the more specific setters. Adjusting settings this way affects **every subsequent** diagram and table unless overridden per-call.

Defaults live in **five** buckets, each with its own setter:

bucket	setter	controls
board	<code>set-board-defaults</code>	square colours, labels, piece set, highlights, arrows, grid, size — full list in Section 10.2
diagram	<code>set-diagram-defaults</code>	the diagram figure: game-info line, supplement, outline title
table	<code>set-table-defaults</code>	the table figure: supplement, outline title, title gap
language	<code>set-lang</code>	the document language (localized strings)
PGN handling	<code>set-pgn-defaults</code>	what a parsed game’s extras render — see Section 5.6

`set-chess-defaults` is the single **umbrella** over **all five** buckets: pass any key from any bucket — **including** `lang` — and it is routed to the bucket that owns it.

```
#set-board-defaults(light: rgb("#eeed2"), dark: rgb("#769656"), size: 5cm)
#set-lang("de")
#set-pgn-defaults(nags: true)
// ...or all of the above at once, through the umbrella:
#set-chess-defaults(dark: blue, lang: "de", nags: true, info-gap: 1em)
#set-piece-set("merida")    // sugar for set-board-defaults(piece-set: ..)
```

Every default is equivalently a per-call argument — the same green theme, set once above vs. passed to one diagram:


```
#chess-diagram(
  "rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/
  RNBQKBNR",
  light: rgb("#eeeeed2"), dark:
  rgb("#769656"),
  size: 4cm,
)
```

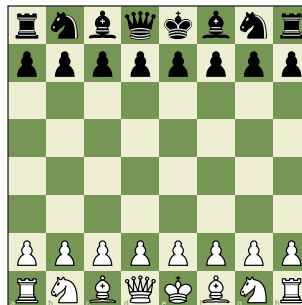


Diagram 9: Position at move 1, White to play

Two caveats. `flip` is **not** allowed in any defaults setter — orientation is a per-board choice, so `set-chess-defaults(flip: ..)` is an error. And `supplement` / `outline-title` live in **both** the diagram and table buckets; the umbrella routes them to **diagram**, so use `set-table-defaults` for the table ones.

8.1 Language

Package **staunton** supports localisation of text-related output. At the moment we support six different languages; apart from the standard English, we offer German, French, Spanish, Italian, Portuguese, and Russian. We can easily extend the list of supported languages by adding new translation files.

The `notation` / `chess-notation` functions localise the piece letters, and the `chess-diagram` / `chess-table` figures carry language-aware titles and captions. The `lang:` argument on each function overrides the document default, and the document default is set with `set-lang`.

A single document **language** drives every language-aware string — diagram and table supplements, outline titles, and notation piece letters. Default is English; `"auto"` follows `#set text(lang: ..)`; or pick a code:

```
#set-lang("de")      // Diagramm / Tabelle / Sf3, Lb5, ...
#set-lang("auto")    // follow #set text(lang: ..)
```

Every localizable string is also per-call overridable (the `lang:` argument seen on `chess-notation` above). Adding a language is a no-code change: drop a `src/assets/il8n/<code>.typ` and register it in `src/il8n.typ`.

8.2 PGN Handling

The fifth bucket, **PGN handling**, decides how much of a parsed game's embedded extras (NAGs, comments, annotations, variations, embedded diagrams) get interpreted at render time. Because those switches are best understood next to the notation and board features they govern, they are documented with the games themselves — see Section 5.6. `set-pgn-defaults` sets them document-wide, and `set-chess-defaults` routes the same keys through the umbrella.

9 HTML Export

Besides paged output (PDF / PNG), **staunton** also supports Typst's **HTML export**. Compile with the `html` feature and target:

```
typst compile --features html --format html your-doc.typ out.html
```

The library-level output maps onto native HTML:

- **move notation** becomes ordinary inline text (mainline moves as `<strong>`);
- **tournament tables** become real `<table>` elements;
- **diagram** and **table** figures keep their captions, numbering and supplements as `<figure>` / `<figcaption>`, and `@`-references and the diagram / table **outlines** resolve to in-document links;
- **boards and diagrams** are embedded as **inline SVG** (a board is layout-drawn, so it is wrapped in `html.frame` under an HTML target — paged export is unchanged).

Caveats. Typst's HTML export is itself *under active development*, so this is **limited** support, not full PDF parity. Page-level chrome (the `#set page` header / footer, `#pagebreak`, multi-column `#grid` layout) is dropped by HTML export — that is document styling, not part of staunton. Treat HTML output as a useful secondary format that will improve as Typst's own HTML support matures.

10 Common Parameters

This chapter collects the recurring **argument value shapes** and the full board / diagram / table option lists — the parameters that many functions share. The **Main functions** and **Behind the scenes** chapters that follow then give every public function’s signature with each parameter’s type and default, generated directly from the source docstrings. The guide chapters above show each feature in use with a rendered example; this reference part answers “what exactly can I pass”.

10.1 Argument Value Shapes

A few arguments accept more than one shape and recur across functions, so they are described once here.

source for `board`, `chess-board`, `diagram`, `chess-diagram`, `position` — a **FEN string** `"rnbqkbnr/..."`; a **position** dict (from `position / parse-fen`); a **squares** dict (`e1: "K"`, `d8: (kind: "q", color: "black")`, `e4: "P"`) (piece as a long name, kind abbreviation, or bare letter — UPPER white, lower black); or the **string form** (rank-per-line rows, `.` = empty; one raw block or several row strings). `chess-*` reject a non-standard **variant**.

locator for `position-after`, `diagram-after`, `to-fen`, `move-san`, `move-node`, and builder addresses — a **mainline** string `"12w"` / `"12b"`, or a **path** dict (`line: (...hops...)`, `at: "<move>"`), each hop (`at: "<move>"`, `into: (descend into variation n at that move)`, to reach a move inside `<n>`)

a (possibly nested) variation. `notation`’s `line:` also takes a bare hops array; its `from / to` are mainline strings only.

size a **length** (default: `4cm`), a **ratio** of the available width (default: `60%`), or **auto**.

highlight entry a square name `"e4"`; a (square, color) pair; or a dict (`square:`, `shape:`, `color:`) with shape `"filled"` / `"cross"` / `"circle"`.

arrows entry (`from, to`); (`from, to, color`); or a dict (`from:`, `to:`, `color:`).

builder overrides for `with-nags` / `with-comments` — a **dict** of mainline locators (`("3w": v)`) or an **array of (locator, value) pairs** (this form also takes path-dict locators). A NAG value is `"$n"` or a suffix glyph `! ? !! ?? !? ?!` (sugar for `$1 – $6`), or an array of those; a comment value is a plain string.

annotation colour letters for PGN `%cal / %csl` and the `annotation-colors` map — `G R Y B O`.

10.2 Board Style Options

Accepted by `board` / `chess-board` / `diagram` / `chess-diagram` per call, and by `set-board-defaults` / `set-chess-defaults` document-wide (see Section 8).

option	default	meaning
<code>size</code>	<code>auto</code>	board size: a length , a ratio of the width, or auto
<code>light / dark</code>	tan theme	the two square fill colours
<code>labels</code>	<code>true</code>	show rank/file labels
<code>label-font</code>	<code>("Arial", "DejaVu Sans Mono")</code>	label font — a family or a fallback list
<code>label-mode</code>	<code>"on-square"</code>	<code>"on-square"</code> / <code>"outside"</code> / <code>"border"</code>
<code>file-side / rank-side</code>	<code>bottom / right</code>	which edge files / ranks sit on
<code>file-label-corner</code> / <code>rank-label-corner</code>	<code>left / right</code>	on-square label corner

option	default	meaning
border-theme	"square"	"border" band theme: "square" / "brown" / "dark"
border	0.5pt + luma(40)	thin board outline (none to drop)
grid	false	1pt grid lines between squares
piece-set	"cburnett"	SVG set name, or "unicode" for the glyph fallback
piece-scale	0.95	fraction of a square a piece occupies
highlight / arrows	()	squares / arrows to draw — see the value shapes
highlight-shape	"filled"	default shape for plain-string highlight entries
highlight-fill / highlight-transparency	green, 75%	filled-highlight colour and its transparency
cross-color / circle-color	red / green	cross / circle stroke colours
cross-width / circle-width	2pt	cross / circle stroke widths
arrow-color / arrow-transparency	green, 85%	default arrow colour and its transparency
arrow-width	auto	arrow shaft width; auto scales with the square
annotation-colors	G/R/Y/B/O map	PGN %cal / %csl colour-letter → colour
label-color	luma(90)	"outside" -mode strip label colour
label-border-ratio	0.07	"border" -mode band width, as a board fraction
white-fill / black-fill / piece-font	—	the "unicode" glyph fallback only
baseline-inset	0.20	glyph fallback: baseline lift (square fraction)

10.3 Diagram, Table, Language, and PGN-Handling Options

Diagram (set-diagram-defaults): info-bold (true), info-gap (0.6em), supplement (auto → localized “Diagram”), outline-title (auto).

Table (set-table-defaults): supplement (auto → “Table”), outline-title (auto), title-gap (0.6em).

Language (set-lang): lang — "en" (default), a code ("de", "es", "fr", "it", "pt", "ru"), or "auto" (follow #set text(lang: ..)).

PGN handling (set-pgn-defaults): annotations, nags, comments, diagrams, variations — all false by default; bold-mainline defaults true (see PGN handling).

11 Main Functions

The everyday API, grouped by task. Each entry is generated from the function's source docstring: its signature, then every parameter with its type and default.

11.1 Boards and Diagrams

board

Draw a bare board — no caption, no figure — the variant-agnostic drawing primitive that every diagram builds on. The variant (if any) rides on `source`; the variant-named wrappers (`chess-board` , a future `xiangqi-board` , ...) are thin sugar over this.

Parameters

```
board(
    source: str | dictionary ,
    flip: bool ,
    ..overrides: arguments
) -> content
```

source `str` or `dictionary`

the position to draw — a **FEN string**, a **position** dict (from `position / parse-fen`), or a bare **squares** dict (`(el: "K", ...)`).

flip `bool`

show the board from Black's side. Per-call only — never a document default.

Default: `false`

..overrides `arguments`

any board **style** option (`size` , `light` , `dark` , `labels` , `label-mode` , `file-side` , `rank-side` , `piece-set` , `highlight` , `arrows` , `grid` , ...) — see Board style options.

chess-board

Standard western-chess board — the variant-named sugar over `board` , and the everyday entry point. Identical rendering, but it documents the variant and rejects a non-standard `source` . (Other variants get their own entry, e.g. a future `xiangqi-board` .)

Parameters

```
chess-board(
    source: str | dictionary ,
    flip: bool ,
    ..overrides: arguments
) -> content
```

source `str` or `dictionary`

a standard-chess position — a FEN string, a position dict, or a squares dict.

flip `bool`

show the board from Black's side.

Default: `false`

..overrides arguments

any board **style** option (as for `board`).

diagram

A board wrapped in a `#figure` – the variant-agnostic diagram, and the generic primitive under `chess-diagram` (and a future `xiangqi-diagram`). Draws an automatic “White – Black (Year)” line above when both players are known, and a default caption below for a FEN source (“Position at move N, X to play”).

Parameters

```
diagram(
    source: str | dictionary ,
    white: str | none ,
    black: str | none ,
    event: str | none ,
    year: int | str | none ,
    caption: auto | content | none ,
    game-info: auto | content | none ,
    flip: bool ,
    lang: auto | str ,
    ..args: arguments
) -> content
```

source str or dictionary

a FEN string, a position dict, or a bare board dict.

white str or none

white player’s name, for the automatic info line.

Default: `none`

black str or none

black player’s name.

Default: `none`

event str or none

event name (carried; not shown by default).

Default: `none`

year int or str or none

year, appended to the info line in parentheses.

Default: `none`

caption (auto) or (content) or (none)

the figure caption; `auto` is the source-specific default (a FEN gets “Position at move N...”; a position or board dict gets none).

Default: `auto`

game-info (auto) or (content) or (none)

the above-board line; `auto` is the automatic player line — pass your own content, or `none` to drop it.

Default: `auto`

flip (bool)

show the board from Black’s side (per-diagram only).

Default: `false`

lang (auto) or (str)

language for the supplement; `auto` follows the document.

Default: `auto`

..args arguments

board **style** options go to the renderer; anything else is forwarded to `#figure` (e.g. `placement: top`).

chess-diagram

Standard western-chess diagram — the variant-named sugar over `diagram` and the everyday entry point. Same behaviour, but it documents the variant and rejects a non-standard `source`.

Parameters

```
chess-diagram(
  source: str dictionary ,
  ..args: arguments
) -> ( content )
```

source (str) or dictionary

a standard-chess position (FEN, position dict, or squares dict).

..args arguments

everything `diagram` accepts (players, caption, game-info, flip, lang, style options, and `#figure` arguments).

11.2 Positions

position

Build a `position` — the data a board draws — from manual placement. (A single string containing `/` is treated as a FEN and delegated to `parse-fen`.) Positional arguments place the pieces; named arguments set the metadata.

Pieces are given as a **squares dict** — `(e1: "K", d8: (kind: "q", color: , a piece being a bare letter "black"))` (UPPER white, lower black) or a `(kind, color)` with a long or abbreviated kind — or in the **string form**, one rank per line (or per positional string), `.` for an empty square.

Parameters

`position(..args: arguments ) -> dictionary`

..args arguments

the piece placement (positional), plus named metadata — `variant` (default "standard"), `turn`, `castling`, `en-passant`, `halfmove`, `fullmove`, and `cols / rows` (default: from the variant, or counted in the string form).

parse-fen

Parse a FEN string into a `position` dict — the inverse of `to-fen`. The result carries the board (square name → `(kind, color)`) plus the side to move, castling rights, en-passant target, and the halfmove / fullmove clocks.

Parameters

`parse-fen(fen: str ) -> dictionary`

fen str

a FEN record, e.g. "rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1".

to-fen

Export a position as a FEN string — the inverse of `parse-fen`. Serialises either a **position** dict directly, or a **game** at a locator. Standard 8×8 positions round-trip exactly.

Parameters

```
to-fen(
  source: dictionary ,
  locator: str dictionary
) -> str
```

source dictionary

a **position** dict, or a **game** (from `parse-pgn`).

locator str or dictionary

required when `source` is a game — which move's position to serialise (a mainline "12w" or a path dict). Ignored for a position.

Default: **none**

starting-fen

The standard chess starting position, as a FEN string constant (handy as the `source` for `chess-moves`, `parse-fen`, `board`, ...).

11.3 Games (PGN)

parse-pgn

Parse PGN text into an array of games. The roster (tags), result, and the verbatim movetext substring are extracted eagerly; the move tree is parsed lazily on first use via `movetext(game)`, so a document that shows only a few positions never parses the rest.

Parameters

```
parse-pgn(input: str content) -> array
```

input str or content

the PGN source — a string, or a raw block (``...``) / `#raw(...)`.

movetext

The parsed movetext tree (an array of move nodes) of a game — the mainline spine with recursive variations. Built on demand from the game's raw movetext and memoised, so deeper structure errors (e.g. a (without a preceding move) surface here.

Parameters

```
movetext(game: dictionary) -> array
```

game dictionary

a parsed game (from `parse-pgn`).

mainline

The mainline of a game as an array of SAN strings (the game as played).

Parameters

```
mainline(game: dictionary) -> array
```

game dictionary

a parsed game (from `parse-pgn`).

game-result

The game result token: "1-0", "0-1", "1/2-1/2", or "*".

Parameters

```
game-result(game: dictionary) -> str
```

game dictionary

a parsed game (from `parse-pgn`).

position-after

The position at a locator — handles the mainline and (nested) variations.

Parameters

```
position-after(
  game: dictionary ,
  locator: str dictionary
) -> dictionary
```

game dictionary
a parsed game (from `parse-pgn`).

locator `str` or dictionary
a mainline `"30w" / "30b"`, or a variation path dict (`line: (...hops...)`, `at: "<move>"`).

diagram-after

A chess diagram for the position at `locator` within a parsed game. Players and year default to the game's roster tags (so the info line is automatic) and the caption defaults to "Position after move ..." (the move played).

When the resolved PGN-handling `annotations` switch is on, `%cal` / `%csl` drawing annotations in the move's comment become arrows / highlights, merged with any passed explicitly.

Parameters

```
diagram-after(
  game: dictionary ,
  locator: str dictionary ,
  white: (auto) str (none) ,
  black: (auto) str (none) ,
  year: (auto) int str (none) ,
  caption: (auto) content (none) ,
  annotations: (auto) bool ,
  flip: bool ,
  game-info: (auto) content (none) ,
  lang: (auto) str ,
  ..args: arguments
) -> content
```

game dictionary
a parsed game (from `parse-pgn`).

locator `str` or dictionary
which position — a mainline `"30w" / "30b"` or a variation path dict.

white `(auto)` or `str` or `(none)`
white player; `auto` uses the game's `White` tag.
Default: `auto`

black `(auto)` or `str` or `(none)`
black player; `auto` uses the `Black` tag.
Default: `auto`

year `(auto)` or `int` or `str` or `(none)`
year; `auto` uses the `Date` tag.
Default: `auto`

caption (auto) or (content) or (none)

auto is "Position after move ...".

Default: auto

annotations (auto) or (bool)

process %cal / %csl into arrows / highlights; auto consults set-pgn-defaults (off by default).

Default: auto

flip (bool)

show the board from Black's side.

Default: false

game-info (auto) or (content) or (none)

the above-board line (as for diagram).

Default: auto

lang (auto) or (str)

language for the supplement; auto follows the document.

Default: auto

..args arguments

board style options and #figure arguments.

move-san

The SAN of the move addressed by locator (e.g. "0-0-0"). Used to build PGN-diagram captions.

Parameters

```
move-san(
  game: dictionary,
  locator: (str) dictionary
) -> (str)
```

game dictionary

a parsed game (from parse-pgn).

locator (str) or dictionary

a mainline "30w" / "30b", or a variation path dict.

move-node

The full move node addressed by locator — its SAN plus nags, comments (comment-before / comment-after) and variations. Used e.g. to recover PGN %cal / %csl annotations for a diagram.

Parameters

```
move-node(
  game: dictionary ,
  locator: (str) dictionary
) -> dictionary
```

game dictionary
a parsed game (from `parse-pgn`).

locator (str) or dictionary
a mainline "30w" / "30b", or a variation path dict.

chess-moves

Apply a run of moves to a position and return the **final** position (renderable). Each move is resolved against the position's legal moves, so an illegal or ambiguous move is a hard error naming the offending SAN token. The variant is taken from `source`; non-standard variants error in the engine.

Parameters

```
chess-moves(
  source: (none) (str) dictionary ,
  moves: (str) (content) array
) -> dictionary
```

source (none) or (str) or dictionary
the starting point — `none` for the standard start, a FEN string, or a position dict.

moves (str) or (content) or array
move text (a string or a raw block; move numbers and a trailing result are tolerated and stripped), or an array of SAN tokens.

11.4 Annotate and Build**with-nags**

Attach NAGs to addressed moves (mainline or inside variations), returning a **new** game (the source is not mutated). Each value **replaces** that move's NAGs.

Parameters

```
with-nags(
  game: dictionary ,
  overrides: dictionary array
) -> dictionary
```

game dictionary
a parsed game (from `parse-pgn`).

overrides dictionary or array
a dict of mainline locators (("12w": "!")) or an array of (locator, value) pairs (a locator may be a variation path dict). A value is "\$n", a suffix glyph (! ? !! ?? !? ?!), or an array of those.

with-comments

Attach text comments to addressed moves (mainline or inside variations), returning a **new** game (the source is not mutated). Each comment **replaces** any existing one and is set as the move's `comment-after` (what notation's `comments` switch renders).

Parameters

```
with-comments(
  game: dictionary,
  overrides: dictionary array
) -> dictionary
```

game dictionary

a parsed game (from `parse-pgn`).

overrides dictionary or array

a dict of mainline locators or an array of `(locator, text)` pairs (a locator may be a variation path dict); each `text` is a plain string.

with-variation

Add a variation (RAV) as an **alternative** to the move at `at`, returning a **new** game (the source is never mutated). The variation is appended to that move's variations (its `into` index is the previous count) and numbered from the move's ply at render time. Legality is checked only if you later navigate into the line (e.g. via `diagram-after`).

Parameters

```
with-variation(
  game: dictionary,
  at: str dictionary,
  moves: str content
) -> dictionary
```

game dictionary

a parsed game (from `parse-pgn`).

at str or dictionary

the move to branch at — a **mainline** locator (`"3w"` / `"3b"`), or a **path** dict (`line: (...hops...)`, `at: "<move>"`) to reach a move inside a (possibly nested) variation.

Default: `none`

moves str or content

a PGN movetext fragment — a plain SAN run like `"Bc4 Bc5"`, or richer text with nested `()`, `$n` NAGs and `{comments}`.

Default: `none`

11.5 Notation

notation

Render move notation as text, and — when `diagrams` is on and the source is a game — splice a `chess-diagram` into the flow after each move whose comment carries a diagram marker (ChessBase `# /`

`#[caption]`, `Scid [d] / [D]`, `\diagram`, `%%diagram`). Embedded diagrams are made in a context, so they are not individually referenceable. The variant-agnostic formatter under `chess-notation`.

Parameters

```
notation(
  source: dictionary | str | array ,
  ..args: arguments
) -> content
```

source dictionary or str or array

a parsed game, a move-text string, or a SAN array.

..args arguments

the formatting options — `from` / `to` (mainline slice), `line` (render one variation), `figurine`, `lang`, `nags`, `comments`, `variations`, `variation-style` ("inline" / "block"), `bold-mainline`, `move-numbers`, `result`, and the embedding switches `diagrams` `annotations`. The auto-defaulting ones consult `set-pgn-defaults`.

chess-notation

Standard western-chess notation — the variant-named sugar over `notation` and the everyday entry point. Same behaviour, but it rejects a non-standard `source` variant.

Parameters

```
chess-notation(
  source: dictionary | str | array ,
  ..args: arguments
) -> content
```

source dictionary or str or array

a parsed game, a move-text string, or a SAN array (standard chess).

..args arguments

everything notation accepts.

11.6 Tournament Tables

standings-table

Render a standings table as a Typst `#table` figure (kind "chess-table"). The data options are exactly as for `standings`.

Parameters

```
standings-table(
  games: array ,
  by: str ,
  tiebreaks: auto | array ,
  match-points: dictionary ,
  title: none | content ,
  caption: none | content ,
  supplement: auto | content ,
  lang: auto | str ,
  ..table-args: arguments
) -> content
```

games array
parsed games (from `parse-pgn`).

by `str`
"player" or "team".
Default: `"player"`

tiebreaks `auto` or array
as for `standings`.
Default: `auto`

match-points dictionary
as for `standings`.
Default: (win: `2`, draw: `1`, loss: `0`)

title `none` or `content`
a title shown above the table.
Default: `none`

caption `none` or `content`
the figure caption (needed for the table to appear in `chess-table-outline`).
Default: `none`

supplement `auto` or `content`
the figure supplement; `auto` is language-aware.
Default: `auto`

lang `auto` or `str`
language for defaults; `auto` follows the document.
Default: `auto`

..table-args arguments
forwarded to `#table`.

crosstable-table

Render a round-robin cross-table as a Typst `#table` figure. Columns are numbered to match the row order; the diagonal is shaded.

Parameters

```
crosstable-table(  
  games: array ,  
  by: str ,  
  match-points: dictionary ,  
  title: (none) | content ,  
  caption: (none) | content ,  
  supplement: (auto) | content ,  
  lang: (auto) | str ,  
  ..table-args: arguments  
) -> content
```

games array

parsed games (from `parse-pgn`).

by str

"player" or "team".

Default: "player"

match-points dictionary

team match-point values.

Default: (win: 2, draw: 1, loss: 0)

title (none) or content

a title shown above the table.

Default: none

caption (none) or content

the figure caption.

Default: none

supplement (auto) or content

the figure supplement; `auto` is language-aware.

Default: auto

lang (auto) or str

language for defaults; `auto` follows the document.

Default: auto

..table-args arguments

forwarded to `#table`.

progress-table

Render a progress table as a Typst `#table` figure: a column per round showing that round's result and the running total, plus a final total.

Parameters

```
progress-table(  
  games: array ,  
  by: str ,  
  match-points: dictionary ,  
  title: (none) | content ,  
  caption: (none) | content ,  
  supplement: (auto) | content ,  
  lang: (auto) | str ,  
  ..table-args: arguments  
) -> content
```

games array

parsed games (from `parse-pgn`).

by str

"player" or "team".

Default: "player"

match-points dictionary

team match-point values.

Default: (win: 2, draw: 1, loss: 0)

title (none) or content

a title shown above the table.

Default: none

caption (none) or content

the figure caption.

Default: none

supplement (auto) or content

the figure supplement; `auto` is language-aware.

Default: auto

lang (auto) or str

language for defaults; `auto` follows the document.

Default: auto

..table-args arguments

forwarded to `#table`.

games-by-event

Split a games array into a dict keyed by the `Event` tag, preserving insertion order within each event. Games without an `Event` tag group under `" "`.

Parameters

`games-by-event`(games: array) -> dictionary

games array

parsed games (from `parse-pgn`).

11.7 Outlines and References

chess-diagram-outline

An outline listing only chess **diagrams** (figures of kind `"chess"`).

Parameters

```
chess-diagram-outline(
  title: (auto) (content) (none),
  lang: (auto) str,
  ..args: arguments
) -> (content)
```

title (auto) or (content) or (none)

the outline title; `auto` is the language-aware “List of Diagrams” (or the document value from `set-diagram-defaults`).

Default: `auto`

lang (auto) or str

language for the default title; `auto` follows the document.

Default: `auto`

..args arguments

forwarded to `outline` (e.g. `depth`, `indent`).

chess-table-outline

An outline listing only chess **tables** (standings / cross-table / progress figures of kind `"chess-table"`).

Only tables that carry a `caption` appear.

Parameters

```
chess-table-outline(
  title: (auto) (content) (none),
  lang: (auto) str,
  ..args: arguments
) -> (content)
```

title (auto) or (content) or (none)

the outline title; auto is the language-aware “List of Tables” (or the document value from set-table-defaults).

Default: auto

lang (auto) or (str)

language for the default title; auto follows the document.

Default: auto

..args arguments

forwarded to outline .

chess-outlines

Print both chess outlines back to back: diagrams first, then tables.

Parameters

```
chess-outlines(
  diagram-title: ( auto ) ( content ) ( none ),
  table-title: ( auto ) ( content ) ( none ),
  lang: ( auto ) ( str ),
  ..args: arguments
) -> ( content )
```

diagram-title (auto) or (content) or (none)

title for the diagram outline; none drops it.

Default: auto

table-title (auto) or (content) or (none)

title for the table outline; none drops it.

Default: auto

lang (auto) or (str)

language for the default titles; auto follows the document.

Default: auto

..args arguments

forwarded to both outline s.

11.8 Document Defaults

set-chess-defaults

Umbrella setter: route each field to the board, diagram, table, i18n or PGN-handling bucket — handy for setting several defaults at once. flip orientation are rejected (per-diagram only).

Parameters

```
set-chess-defaults(..fields: arguments ) -> content
```

..fields arguments

any named style option from the board, diagram, table, i18n or PGN-handling groups; unknown keys error. (`supplement` `outline-title` , shared by diagram and table, route to the diagram bucket — use `set-table-defaults` for the table ones.)

set-board-defaults

Set default **board** style fields for all subsequent boards and diagrams (document-order state, like a Typst `#set`). `flip` is rejected (per-diagram only).

Parameters

```
set-board-defaults(..fields: arguments ) -> content
```

..fields arguments

named board style options (`size` , `light` , `dark` , `labels` , `label-mode` , `piece-set` , ...); unknown keys error.

set-diagram-defaults

Set default **diagram** style fields (the `#figure` wrapper) for subsequent diagrams.

Parameters

```
set-diagram-defaults(..fields: arguments ) -> content
```

..fields arguments

named diagram style options (`supplement` , `outline-title` , `info-bold` , `info-gap` , ...); unknown keys error.

set-table-defaults

Set default **table** style fields (the `#figure` wrapper around tournament tables) for subsequent `*-table` renderers.

Parameters

```
set-table-defaults(..fields: arguments ) -> content
```

..fields arguments

named table style options — currently `supplement` (default “Table”), `outline-title` , `title-gap` ; unknown keys error.

set-pgn-defaults

Set default PGN-handling fields for subsequent notation / diagram output.

Parameters

```
set-pgn-defaults(..fields: arguments ) -> content
```

..fields arguments

named switches — `annotations` , `nags` , `comments` , `diagrams` , `variations` , `bold-mainline` ; unknown keys error.

set-lang

Set the document language for all language-aware strings (supplements, outline titles, notation piece letters).

Parameters

```
set-lang (code: str) -> content
```

code str

a language code ("en" , "de" , ...) or "auto" (follow #set text(lang: ..)).

set-piece-set

Set the default piece set for subsequent diagrams — a convenience wrapper over `set-board-defaults(piece-set: name)`.

Parameters

```
set-piece-set (name: str) -> content
```

name str

a bundled piece-set name ("cburnett" , "merida" , ...) or a registered custom set.

12 Behind the Scenes

Lower-level public primitives — usable, but not the everyday entry points. Private helpers (`_`-prefixed functions) are intentionally omitted.

12.1 Engine and Position Internals

legal-moves

All fully-legal moves for the side to move in `position`, as an array of move dicts.

Parameters

```
legal-moves(position: dictionary) -> array
```

position dictionary
the position to generate moves for.

apply

Apply a move to a position, returning the new position — pure (the input is not mutated). Handles captures, en passant, castling (rook too), promotion, castling-rights and en-passant-target updates, and the move clocks.

Parameters

```
apply(  
    position: dictionary,  
    move: dictionary  
) -> dictionary
```

position dictionary
the position to move from.

move dictionary
a concrete move dict (e.g. from `san-to-move` or `legal-moves`).

in-check

Is `color`'s king currently in check in `position`?

Parameters

```
in-check(  
    position: dictionary,  
    color: str  
) -> bool
```

position dictionary
the position to test.

color str
"white" or "black".

position-fen

Encode a `position` dict back into a FEN string (the inverse of `parse-fen`). Geometry-aware — it uses the position's `cols` / `rows`, so it also serialises larger-than-8×8 boards — and tolerant of positions built

by `position()` (empty `castling`, `en-passant` `none`). Standard 8×8 positions round-trip exactly with `parse-fen`.

Parameters

```
position-fen(position: dictionary) -> str
```

position dictionary

a position dict (from `position` / `parse-fen`).

game-start

The starting position of a game — from its `FEN` tag if present, else the standard start.

Parameters

```
game-start(game: dictionary) -> dictionary
```

game dictionary

a parsed game (from `parse-pgn`).

12.2 SAN

san-to-move

Resolve a SAN string against a position's legal moves, returning the concrete move dict. Illegal or ambiguous SAN is a hard error.

Parameters

```
san-to-move(  
    position: dictionary ,  
    san: str  
) -> dictionary
```

position dictionary

the position to resolve the move in.

san str

a SAN token, e.g. "Nf3", "exd5", "0-0", "e8=Q+".

play-san

Resolve a SAN string and return the resulting position — a convenience wrapper for `apply(position, san-to-move(position, san))`.

Parameters

```
play-san(  
    position: dictionary ,  
    san: str  
) -> dictionary
```

position dictionary

the position to play the move in.

san `str`

a SAN token.

12.3 Tournament Data

standings

Compute standings — an array of records sorted best-first: (rank, name, score, played, wins, draws, losses, buchholz, sonneborn-berger, [board-points for teams])

Parameters

```
standings(
  games: array ,
  by: str ,
  tiebreaks: (auto) array ,
  match-points: dictionary
) -> array
```

games array

parsed games (from `parse-pgn`).

by `str`

"player" or "team".

Default: `"player"`

tiebreaks `(auto)` or array

ordered tiebreak keys, or `auto` for the mode default.

Default: `auto`

match-points dictionary

team match-point values, (win:, draw:, loss:).

Default: (win: 2, draw: 1, loss: 0)

crosstable

Cross-table for a round-robin: (names, ranks, totals, matrix), rows/cols in standings order, `matrix.at(i).at(j)` entity i's score vs j (player: game points; team: board points), `none` on the diagonal. Errors if the entity does not form a round-robin (some pair never met) — use `standings + progress` for Swiss / league events.

Parameters

```
crosstable(
  games: array ,
  by: str ,
  match-points: dictionary
) -> dictionary
```


games array
 parsed games (from `parse-pgn`).

by `str`
 "player" or "team" .
 Default: `"player"`

match-points dictionary
 team match-point values.
 Default: (win: `2`, draw: `1`, loss: `0`)

progress

Per-entity round-by-round progress: `(names, ranks, rounds, cells)`, where `cells.at(i).at(k)` is `(score, cumulative)` for entity `i` in `rounds.at(k)` (player: game points that round; team: match points). Needs the `Round` tag; works for open / Swiss events too.

Parameters

```
progress(  
  games: array ,  
  by: str ,  
  match-points: dictionary  
) -> dictionary
```

games array
 parsed games (from `parse-pgn`).

by `str`
 "player" or "team" .
 Default: `"player"`

match-points dictionary
 team match-point values.
 Default: (win: `2`, draw: `1`, loss: `0`)

12.4 Rendering and Annotation

render-board

Render a board (square name → `(kind, color)`) to content — the low-level drawing primitive under board. `flip: true` puts Black at the bottom.

Parameters

```
render-board(
  squares: dictionary,
  flip: bool,
  cols: int,
  rows: int,
  ..overrides: arguments
) -> content
```

squares dictionary
square name → (kind, color) .

flip bool
show the board from Black's side.
Default: false

cols int
board width (default 8).
Default: 8

rows int
board height (default 8).
Default: 8

..overrides arguments
named board style fields (see the Board style options).

interpret-comment

Interpret a PGN move comment: extract a diagram marker, %cal arrows, %csl highlights, and the remaining prose. Returns (diagram, arrows, highlights, .text)

Parameters

```
interpret-comment(comment: str or none) -> dictionary
```

comment str or none
the raw comment string, or none .

12.5 Coordinates**file-letter**

The file letter for a zero-based column index (0 → "a", 25 → "z").

Parameters

```
file-letter(col: int) -> str
```

col `int`

the column index (0–25).

parse-square

Parse an algebraic square name into zero-based indices, bounds-checked against the board geometry. One leading file letter (a–z) plus a 1+ digit rank; capitalisation does not matter ("E4" = "e4"). Returns `(col, row)`.

Parameters

```
parse-square(  
    square: str,  
    cols: int,  
    rows: int  
) -> dictionary
```

square `str`

the square name, e.g. "e4" .

cols `int`

board width (default 8).

Default: 8

rows `int`

board height (default 8).

Default: 8

square-name

Inverse of `parse-square` : `(col, row) → "e4"` . Geometry-agnostic (the indices already encode the position); validates only against the 26-file alphabet.

Parameters

```
square-name(  
    col: int,  
    row: int  
) -> str
```

col `int`

zero-based column index.

row `int`

zero-based row index.

is-dark-square

Is the square dark? `a1 = (0, 0)` is a dark square in standard orientation.

Parameters

```
is-dark-square(  
  col: int,  
  row: int  
) -> bool
```

col int

zero-based column index.

row int

zero-based row index.

file-letters

The standard 8×8 file letters ("a", ..., "h") — kept for the (8×8) renderer and callers that want the classic set.

12.6 Pieces and variants

piece-content

Render a single piece as `size`-tall content (no square background) — the SVG piece if the active set has one, else the glyph fallback.

Parameters

```
piece-content(  
  kind: str,  
  color: str,  
  size: length,  
  white-fill: color,  
  black-fill: color,  
  stroke-ratio: float,  
  font: array  
) -> content
```

kind str

one of piece-kinds .

color str

"white" or "black" .

size length

the glyph font size.

white-fill color

fill for white pieces.

Default: default-white-fill

black-fill (color)

fill for black pieces.

Default: default-black-fill

stroke-ratio (float)

outline width as a fraction of size .

Default: 0.03

font array

the glyph-fallback font family list.

Default: default-piece-fonts

fen-piece

Map a single FEN piece letter to (kind, color) — uppercase = white, lowercase = black (e.g. "N" → knight/white, "q" → queen/black).

Parameters

fen-piece(letter: str) -> dictionary

letter (str)

a one-character FEN piece letter.

variant-spec

The spec dict for a variant; errors on an unknown variant name.

Parameters

variant-spec(variant: str) -> dictionary

variant (str)

a variant name (a key of variants).

char-to-piece

Map a single piece character to (kind, color) for a variant — uppercase = white, lowercase = black. Errors on an unknown abbreviation.

Parameters

```
char-to-piece(  
    ch: str,  
    variant: str  
) -> dictionary
```

ch (str)

a one-character piece abbreviation.

variant `str`

the variant whose abbreviations to use (default "standard").

Default: "standard"

piece-kinds

The six chess piece kinds: king, queen, rook, bishop, knight, pawn.

piece-colors

The two piece colours: white and black.

default-piece-set

The factory-default piece set ("cburnett"). Set "unicode" (or none) to select the glyph fallback.

known-piece-sets

The piece sets bundled with the package ("cburnett", "merida") — an advisory list, not a guard: any set name is accepted (drop a folder under `src/assets/piece_sets/<name>/`).

variants

The variant registry: a dict of variant name → spec (name, kinds, abbr, cols, . Only standard is implemented today)

12.7 Localization

resolve-lang

Resolve an effective language code. Must be called inside a `context` . `call-lang == auto` uses the document `lang` setting; "auto" follows `text.lang` ; any other code is used as given.

Parameters

```
resolve-lang(call-lang: auto str) -> str
```

call-lang `auto` or `str`

the per-call language, or `auto` for the document default.

ui-string

Look up one localized UI string (English fallback for an unknown code or key). Must be called inside a `context` .

Parameters

```
ui-string(
  call-lang: auto str,
  key: str
) -> str
```

call-lang `auto` or `str`

the language, or `auto` for the document default.

key `str`

the string key (e.g. "diagram-supplement").